СИСТЕМНЫЙ ДИЗАЙН · ДОМАШНЕЕ ЗАДАНИЕ

Публикация ровно один раз

200 000 SEO-статей в день на сайты клиентов:
Shopify, WordPress, Webflow, Wix

ЗАДАЧА

Что нужно сделать

200 000

статей в день на сайты клиентов

2,3 в секунду в среднем и около 25 в пике, когда все ставят публикацию на 09:00

Любой вызов API может упасть

Таймаут, 5xx, обрыв сети. И непонятно, сработал ли вызов.

Сервис могут перезапустить

В любой момент, посреди публикации.

Каждая статья выходит ровно один раз

После перезапуска: без дублей и без пропусков.

ДАННЫЕ

Статья: текст и картинки

Проблема. У статьи есть текст (10–30 КБ) и картинки (до 3 МБ). Картинки в базе раздувают её и тормозят бэкапы, а текст в отдельном хранилище — лишний запрос на каждую статью.

Решение. Текст храним в Postgres: воркер получит его тем же запросом, которым берёт задачу. Картинки кладём в S3, а в базе держим только ссылку на файл.



200 000 статей × 20 КБ ≈ 4 ГБ текста в день. Для Postgres это нормально: большие поля он сжимает и хранит отдельно от строки.

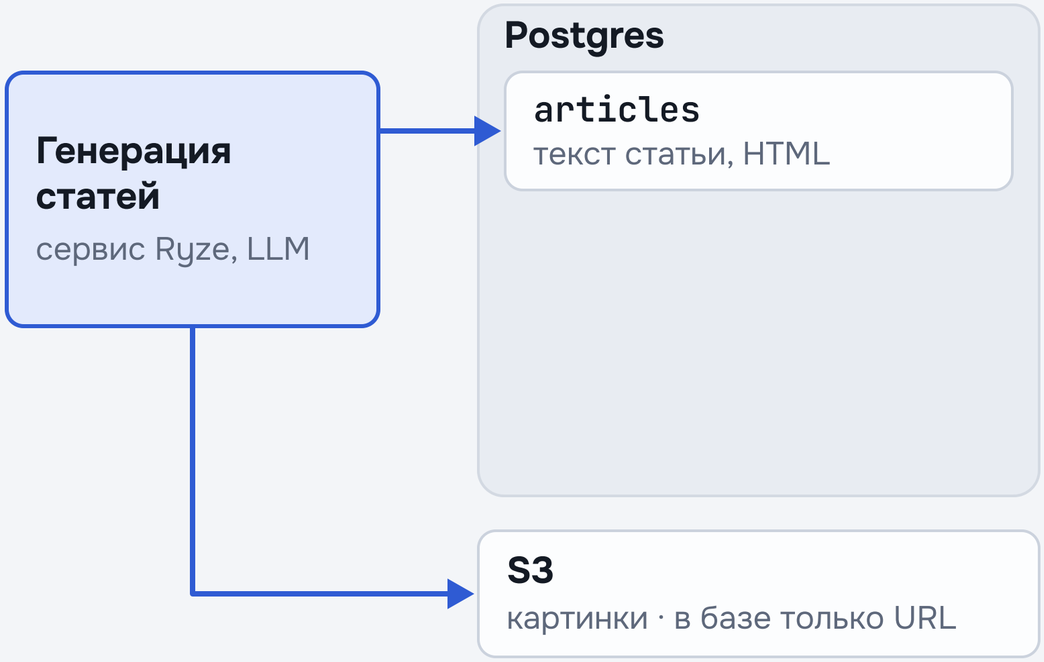
Картинки бинарные и тяжёлые. Им место в объектном хранилище, а отдавать их удобно через CDN.

ДАННЫЕ

Откуда берутся статьи

Проблема. Статьи пишет отдельный сервис генерации. Нужно договориться, в какой момент статья «готова» и с чего начинается наша работа.

Решение. Генерация кладёт текст в Postgres, картинки в S3 и даёт статье постоянный uid. Как только статья записана, её публикация — наша забота.



Генерация — уже существующий сервис. Мы его не трогаем, только договариваемся о формате: у каждой статьи есть стабильный **uid**.

Этот uid потом станет маркером на опубликованном посте. По нему мы узнаем свой пост на сайте клиента.

ТРЕБОВАНИЯ

Что должна уметь система

А Ровно один раз

Каждая статья выходит на сайт один раз: без дублей и без пропусков.

Б Сбой не теряет работу

Таймаут, 5xx, рестарт, падение воркера: продолжаем с того же места.

В Лимиты каждого сайта

Не превышаем лимиты платформ, и большой клиент не тормозит остальных.

Г Новая платформа без переделок

Shopify, WordPress, Webflow, Wix и следующая: ядро о них не знает.

200 000 в день, пик ~25/с

статья выходит за ~15 минут

клиент видит статус каждой статьи

ГЛАВНАЯ ТРУДНОСТЬ

Мы отправили «создай пост», а ответ не пришёл.

Пост создан или нет, узнать нельзя. Повторим запрос, и может выйти дубль. Не повторим, и статья может не выйти. Ключей идемпотентности у этих платформ нет.

Поэтому цель другая: не «вызвать API один раз», а «читатель увидел статью один раз».

ШАГ 1 · ОЧЕРЕДЬ

Очередь задач в той же базе

Проблема. Нужно помнить, какие статьи на какие сайты ещё не вышли. Этот список должен пережить рестарт, а повторный запуск дневного батча не должен создать дубли.

Решение. Таблица publications: одна строка — одна статья для одного сайта, вместе с её статусом. Пара (статья, сайт) уникальна, поэтому повторная вставка ничего не добавит.



Генерация ставит задачу через **INSERT … ON CONFLICT DO NOTHING**: «вставь строку, а если такая пара уже есть, молча пропусти».

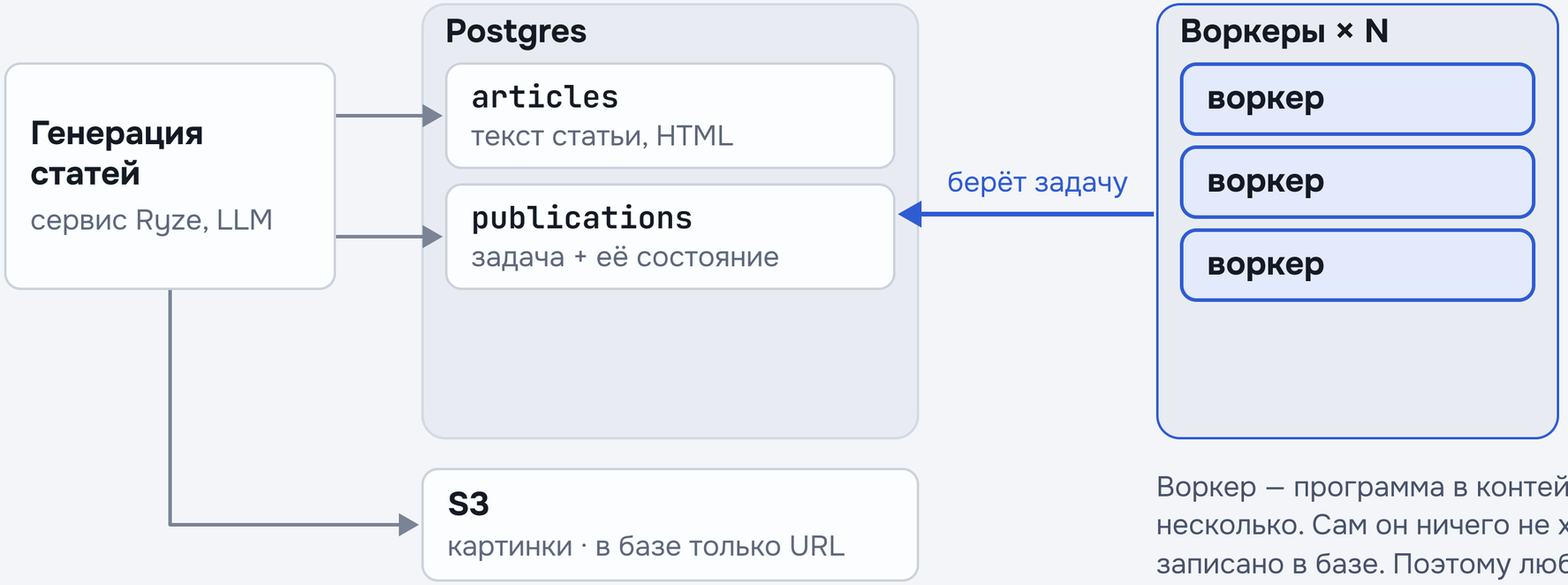
Почему не Kafka: сообщение в очереди нельзя обновить («шаг 2 готов, пост 42»). Прогресс всё равно пришлось бы хранить в базе, и стало бы две системы, которые надо держать согласованными.

ШАГ 2 · ВОРКЕРЫ

Воркеры забирают задачи

Проблема. Кто-то должен брать задачи и ходить в API сайтов. Если он упадёт посреди работы, задача не должна навсегда зависнуть со статусом «в работе».

Решение. Беря задачу, воркер сразу ставит ей время повтора: «через 30 с». Справился — пишет «готово». Умер — время наступит, и задачу заберёт другой.



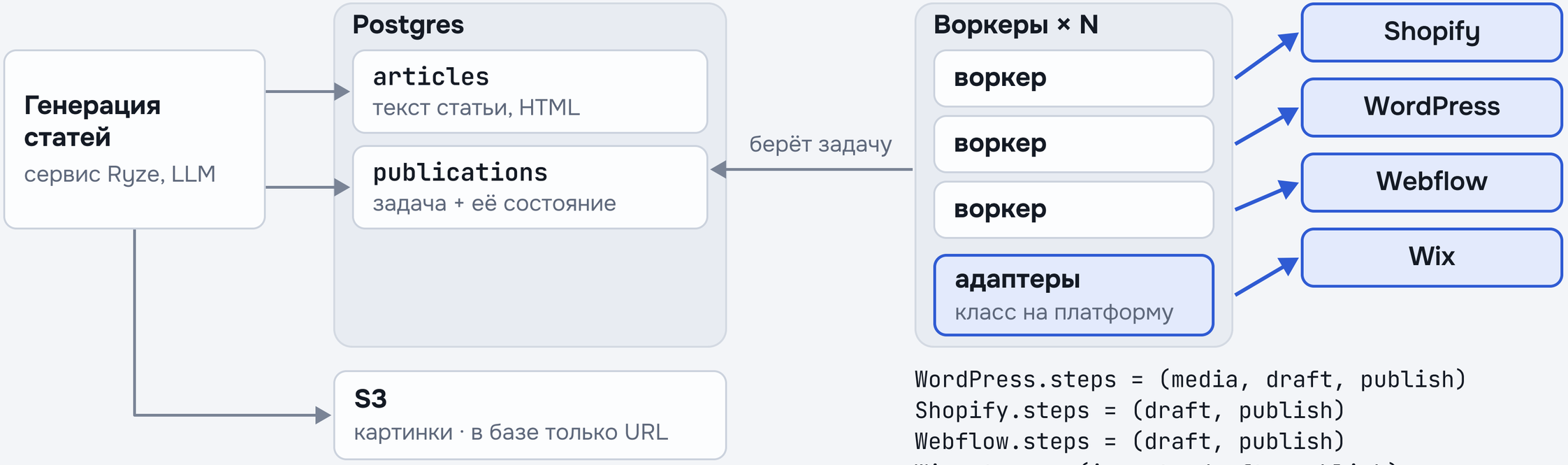
Воркер — программа в контейнере, их запущено несколько. Сам он ничего не хранит: всё сделанное записано в базе. Поэтому любой воркер можно убить, и другой продолжит.

ШАГ 3 · ПЛАТФОРМЫ

Адаптер на каждую платформу

Проблема. У каждой платформы свой API: свои запросы, свой порядок действий, свои ошибки. Если писать это в общем коде воркера, каждая новая платформа будет его ломать.

Решение. Всё платформенное живёт в адаптере: он объявляет свои шаги (steps) и выполняет их. Воркер просто идёт по шагам адаптера и о платформах не знает.




```
WordPress.steps = (media, draft, publish)
Shopify.steps = (draft, publish)
Webflow.steps = (draft, publish)
Wix.steps = (import, draft, publish)
```

ШАГ 4 · ПРОГРЕСС

Запоминаем, где остановились

Проблема. Публикация — это несколько запросов к сайту подряд. Если воркер упадёт после второго, а мы ничего не запомнили, новый начнёт сначала: снова загрузит картинку и создаст второй пост.

Решение. После каждого шага пишем в строку статьи, что уже сделано и что вернул сайт. Новый воркер читает строку и продолжает с того же шага.

```
WordPress.steps = (media, draft, publish)
```

шаг 1 · media

Загрузить картинку

в базу: step = draft, media_id = 17

→

шаг 2 · draft

Создать черновик

в базу: step = publish, post_id = 42

→

шаг 3 · publish

Опубликовать пост 42

в базу: status = done

Воркер упал после шага 2

В строке уже записано step = publish и post_id = 42. Новый воркер сразу делает шаг 3: картинку заново не грузит, второй пост не создаёт.

За статью взялись два воркера

Запись идёт с условием «только если шаг ещё draft» (compare-and-set). Первый воркер записывает, у второго условие уже не выполняется, и он уходит.

```
UPDATE … WHERE step = 'draft'
```

ШАГ 5 · ВЕСЬ ЦИКЛ

Пайплайн воркера

Что делает воркер	Что пишет в базу
1. Арендует сайт клиента	сайт: lease_until = +60 с
2. Берёт статью этого сайта, у которой наступил run_at	attempts + 1, run_at = + backoff
3. media: загружает картинку на сайт	step = draft, media_id
4. draft: ищет свой пост по маркеру, нет — создаёт черновик без slug	step = publish, post_id
5. publish: лишние черновики в корзину, публикует пост со slug	status = done, url
6. Берёт следующую статью; через 20 с отдаёт сайт	сайт: lease_until = сейчас

Пока идут шаги, статья в **status = pending**. Каждая запись шага идёт с условием **WHERE step = текущий** (CAS).

Если что-то пошло не так

Таймаут, 5xx
last_error; повтор сам, когда наступит run_at

429 «слишком часто»
attempts − 1; сайт на паузу до Retry-After

401 / 403
attempts − 1; сайт: status = needs_reauth

4xx или кончились попытки
status = failed

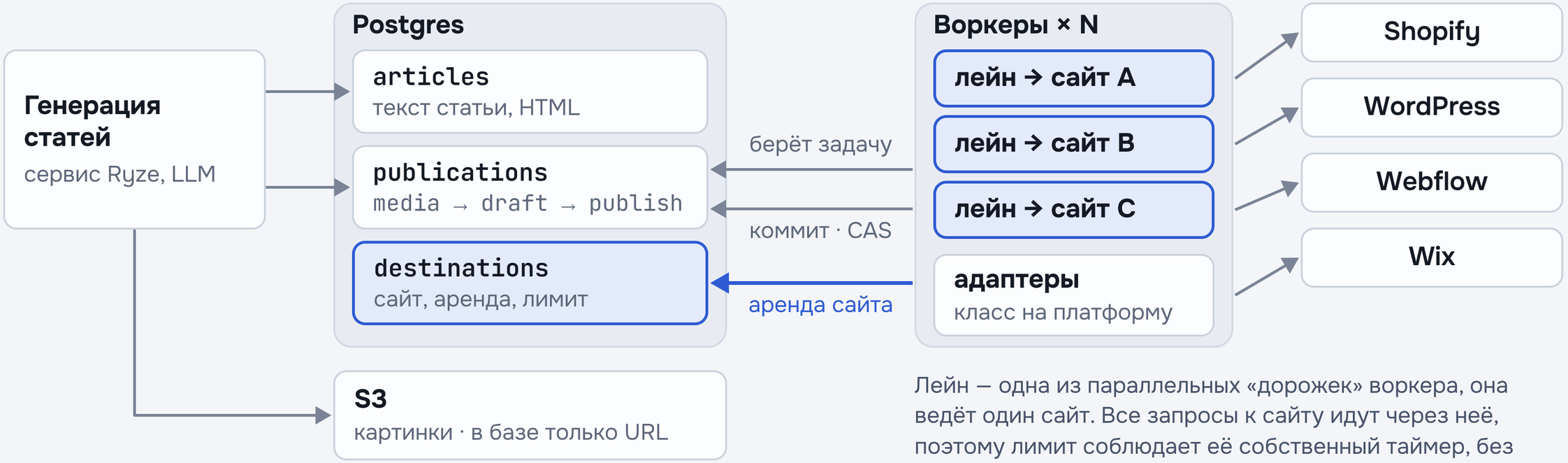
CAS обновил 0 строк
шаг уже записал другой воркер, этот уходит

ШАГ 6 · ЛИМИТЫ

Аренда сайта, а не статьи

Проблема. У каждого сайта лимит запросов, у Shopify около 2 в секунду. Если в сайт разом пойдут 20 воркеров, он ответит «слишком часто», а другие клиенты будут ждать.

Решение. Воркер «арендует» сайт: пишет в базу «сайт мой до 12:00:30» и работает с ним один, с паузой между запросами. Через 20 с отдаёт сайт следующему.

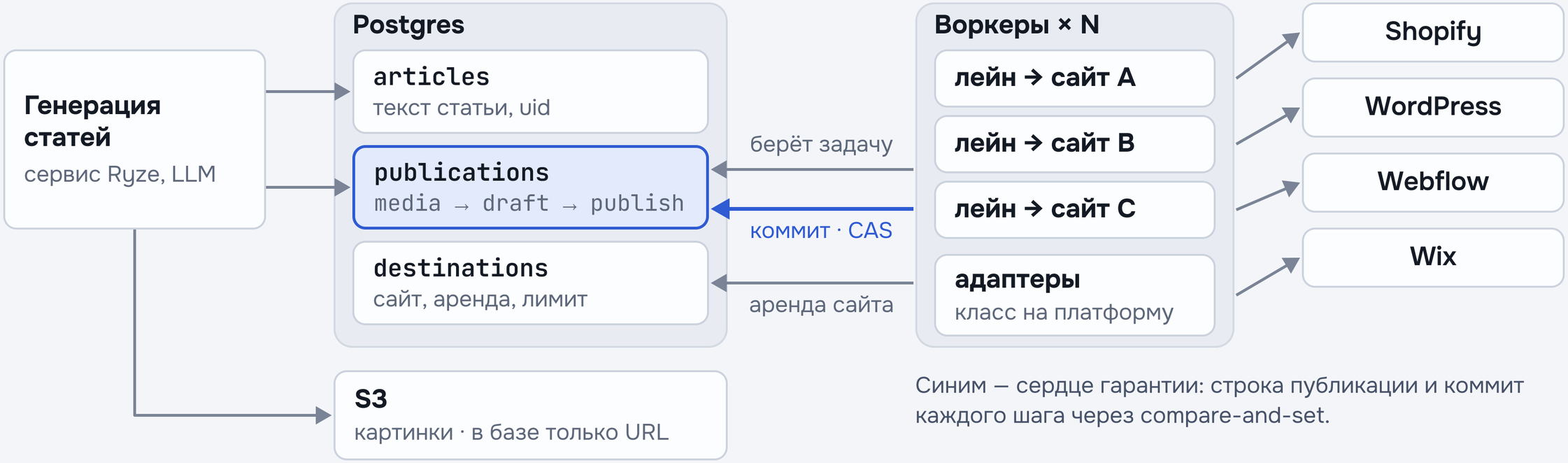


Лейн — одна из параллельных «дорожек» воркера, она ведёт один сайт. Все запросы к сайту идут через неё, поэтому лимит соблюдает её собственный таймер, без общего счётчика в Redis.

ИТОГ

Вся система

Три таблицы, один цикл воркера и адаптер на каждую платформу. Никаких Kafka, Redis и Temporal.



Синим — сердце гарантии: строка публикации и коммит каждого шага через compare-and-set.

РЕПОЗИТОРИЙ

Что лежит в коде

github.com/MaksimovDania/exactly-once-publisher

publisher.py
Воркер, ~150 строк: аренда сайта, взятие статьи, шаги адаптера, коммит через CAS, повторы и паузы.

platforms.py
Контракт адаптера, перевод ошибок в 4 вида, темп сайта и адаптер WordPress.

schema.sql
Три таблицы: articles, publications, destinations.

fake_platform.py
Поддельный WordPress, который нарочно ломается: ошибки, зависания, 429, медленный поиск.

chaos.py
Сквозной тест: гоняет воркеры под хаосом и проверяет, что каждая статья вышла ровно один раз.

README.md · REVIEW.md
Технический дизайн со схемами и независимое ревью с найденными и исправленными багами.

```
docker compose up -d && python chaos.py
```

ТЕСТИРОВАНИЕ

Как я проверял

Chaos-тест

20 сайтов, 500 статей, очередь заполнена дважды. Каждые 1–3 с воркер убивают kill -9 или замораживают дольше аренды.

Сайт, который ломается

Фейковый WordPress: 10% ошибок до и после записи, зависания, 429, отстающий поиск, правила slug.

Гонка двух воркеров

Оба стартуют с одного снимка и создают по черновику. Опубликован ровно один, второй уходит.

Восстановление из бэкапа

Строку статьи откатывают, потом теряют и ставят заново. Пост находится по uid, дубля нет.

Намеренные поломки

Убрал CAS, вернул slug в черновик и ещё 3 правки. Тест падает на каждой из пяти.

Независимое ревью

13 жёстких прогонов, до 50% сбоев и обрывы базы. Исправления перепроверены на настоящем WordPress 7.1.3.

В конце каждого прогона проверка одна: каждая статья live ровно один раз и под своим адресом.

РЕЗУЛЬТАТ

Что делали и что получили

1. **Сгенерировали 500 статей** для 20 сайтов. У каждой десятой адрес на кириллице.
2. **Поставили их в очередь дважды**, как будто дневной батч запустили повторно.
3. **Подняли фейковый WordPress, который ломается:** 10% запросов падают до записи, после неё или зависают дольше таймаута.
4. **Задали лимиты:** сайт держит 10 запросов в секунду, а часть сайтов мы настроили на 20, и они получают 429.
5. **Запустили 4 воркера по 8 лейнов** и каждые 1–3 с убивали один (kill -9) или замораживали дольше аренды. У одного сайта отозвали пароль и через 15 с вернули.
6. **Проверили каждую статью на сайте:** вышла ли она и сколько раз.

ИТОГ

500 / 500

статей вышли ровно один раз и под своим адресом: 0 дублей, 0 пропусков

25 + 7 воркеров убито и заморожено, работа не потерялась

163 ответа 429: сайт вставал на паузу, попытки не сгорали

0 попыток потратил сайт с отозванным паролем, пока ждал

Отдельно тест специально воспроизводит гонку двух воркеров за одну статью и восстановление базы из бэкапа. Все 5 намеренных поломок кода он поймал.